EKF-SLAM ROS2 Project Report

Updated Implementation Analysis

Jaisel Singh, Huan Gu and Qinghua He
Columbia University
Department of Mechanical Engineering

December 14, 2025

Abstract

This report provides a comprehensive analysis of the `EKF_SLAM_ROS2` repository, examining its structure, theoretical foundations, and implementation details. We present complete mathematical derivations for Extended Kalman Filter-based Simultaneous Localization and Mapping (EKF-SLAM), compare the repository's implementation against the theoretical framework, and identify areas for improvement. The analysis covers motion models, measurement models, state estimation, and the critical Jacobian computations that enable linearization of the nonlinear SLAM problem.

The repository implements two variants of EKF-SLAM in Python for ROS2:

1. **Feature-based SLAM** using Split-and-Merge line segment detection for corner extraction
2. **Clustering-based SLAM** using DBSCAN clustering for point landmark detection

Both implementations include real-time state estimation, landmark management, data association, and occupancy grid mapping for visualization.

Contents

1 Introduction

Simultaneous Localization and Mapping (SLAM) is a fundamental problem in mobile robotics where a robot must construct a map of an unknown environment while simultaneously tracking its own location within that map. The Extended Kalman Filter (EKF) approach to SLAM remains one of the most well-understood and theoretically grounded solutions to this problem.

1.1 Project Overview

The `EKF_SLAM_ROS2` repository implements an EKF-SLAM system within the Robot Operating System 2 (ROS2) framework. The project aims to provide:

- Real-time state estimation for mobile robot localization
- Landmark-based mapping using sensor observations
- Integration with ROS2's navigation stack
- Visualization tools for debugging and analysis
- Two complementary landmark detection approaches

1.2 Repository Structure

The repository follows standard ROS2 Python package conventions:

```
EKF_SLAM_ROS2/
        src/
              ekf_slam/
                    ekf_slam_node.py           # Feature-based EKF SLAM (Split-and-Merge)
                    ekf_slam_clustering_node.py # Clustering-based EKF SLAM (DBSCAN)
                    __init__.py
        launch/                             # Launch files for different configurations
        rviz/                               # RViz visualization configurations
        worlds/                             # Gazebo world files
        config/                             # Navigation and parameter files
        doc/
              report/                         # Technical documentation
        README.md
```

Listing 1: Repository Directory Structure

2 Theoretical Background: EKF-SLAM

2.1 State Representation

In EKF-SLAM, the state vector $\boldsymbol{x}_t$ at time t contains both the robot pose and all observed landmark positions:

$$\boldsymbol{x}_t = \begin{bmatrix} \boldsymbol{x}_t^{(r)} \\ \boldsymbol{x}_t^{(m)} \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \\ \theta_t \\ m_{1,x} \\ m_{1,y} \\ \vdots \\ m_{n,x} \\ m_{n,y} \end{bmatrix} \in \mathbb{R}^{3+2n} \tag{1}$$

where:

- (x_t, y_t, θ_t) represents the robot pose (position and orientation)
- $(m_{i,x}, m_{i,y})$ represents the position of the i-th landmark
- n is the total number of observed landmarks

The associated covariance matrix has the block structure:

$$\boldsymbol{P}_t = \begin{bmatrix} \boldsymbol{P}_{rr} & \boldsymbol{P}_{rm} \\ \boldsymbol{P}_{mr} & \boldsymbol{P}_{mm} \end{bmatrix} \tag{2}$$

where $\boldsymbol{P}_{rr} \in \mathbb{R}^{3\times3}$ captures robot pose uncertainty, $\boldsymbol{P}_{mm} \in \mathbb{R}^{2n\times2n}$ captures landmark position uncertainties, and $\boldsymbol{P}_{rm}$, $\boldsymbol{P}_{mr}$ capture the correlations between robot and landmark estimates.

2.2 The EKF-SLAM Algorithm

The EKF-SLAM algorithm consists of two main phases that execute recursively:

Algorithm 1 EKF-SLAM Algorithm

Require: Initial state $\bar{\boldsymbol{x}}_0$, initial covariance $\bar{\boldsymbol{P}}_0$
1: **for** each time step $t = 1, 2, \ldots$ **do**
2: **Prediction Step:**
3: Predict state: $\bar{\boldsymbol{x}}_t = g(\boldsymbol{x}_{t-1}, \boldsymbol{u}_t)$
4: Predict covariance: $\bar{\boldsymbol{P}}_t = \boldsymbol{G}_t \boldsymbol{P}_{t-1} \boldsymbol{G}_t^\top + \boldsymbol{R}_t$
5: **Update Step:** (for each observation $\boldsymbol{z}_t^i$)
6: Compute Kalman gain: $\boldsymbol{K}_t = \bar{\boldsymbol{P}}_t \boldsymbol{H}_t^\top (\boldsymbol{H}_t \bar{\boldsymbol{P}}_t \boldsymbol{H}_t^\top + \boldsymbol{Q}_t)^{-1}$
7: Update state: $\boldsymbol{x}_t = \bar{\boldsymbol{x}}_t + \boldsymbol{K}_t(\boldsymbol{z}_t^i - h(\bar{\boldsymbol{x}}_t))$
8: Update covariance: $\boldsymbol{P}_t = (\boldsymbol{I} - \boldsymbol{K}_t \boldsymbol{H}_t)\bar{\boldsymbol{P}}_t$
9: **end for**

3 Motion Model Derivation

3.1 Velocity Motion Model

The repository implements a velocity-based motion model, which is common for differential-drive and car-like robots. Given control inputs $\boldsymbol{u}_t = (v_t, \omega_t)^\top$ representing linear and angular velocities:

$$g(\boldsymbol{x}_{t-1}, \boldsymbol{u}_t) = \begin{bmatrix} x_{t-1} + v_t \Delta t \cos(\theta_{t-1} + \omega_t \Delta t/2) \\ y_{t-1} + v_t \Delta t \sin(\theta_{t-1} + \omega_t \Delta t/2) \\ \theta_{t-1} + \omega_t \Delta t \end{bmatrix} \tag{3}$$

The midpoint integration $(\theta_{t-1} + \omega_t \Delta t/2)$ provides better accuracy than simple Euler integration by approximating the average heading during the motion interval.

3.2 Motion Model Jacobian

For the EKF, we require the Jacobian of the motion model with respect to the state. Since landmarks are assumed stationary, the full Jacobian is:

$$\boldsymbol{G}_t = \frac{\partial g}{\partial \boldsymbol{x}} = \begin{bmatrix} \boldsymbol{G}_t^{(r)} & \boldsymbol{0}_{3\times2n} \\ \boldsymbol{0}_{2n\times3} & \boldsymbol{I}_{2n} \end{bmatrix} \tag{4}$$

where the robot-specific Jacobian is:

$$\boldsymbol{G}_t^{(r)} = \frac{\partial g}{\partial \boldsymbol{x}^{(r)}} = \begin{bmatrix} 1 & 0 & -v_t \Delta t \sin(\theta_{t-1} + \omega_t \Delta t/2) \\ 0 & 1 & v_t \Delta t \cos(\theta_{t-1} + \omega_t \Delta t/2) \\ 0 & 0 & 1 \end{bmatrix} \tag{5}$$

3.3 Process Noise

The process noise covariance $\boldsymbol{R}_t$ models uncertainty in the motion model. A common formulation maps control-space noise to state-space:

$$\boldsymbol{R}_t = \boldsymbol{V}_t \boldsymbol{M}_t \boldsymbol{V}_t^\top \tag{6}$$

where $\boldsymbol{V}_t = \frac{\partial g}{\partial \boldsymbol{u}}$ is the Jacobian with respect to controls:

$$\boldsymbol{V}_t = \begin{bmatrix} \Delta t \cos(\theta_{t-1} + \omega_t \Delta t/2) & -\frac{v_t \Delta t^2}{2} \sin(\theta_{t-1} + \omega_t \Delta t/2) \\ \Delta t \sin(\theta_{t-1} + \omega_t \Delta t/2) & \frac{v_t \Delta t^2}{2} \cos(\theta_{t-1} + \omega_t \Delta t/2) \\ 0 & \Delta t \end{bmatrix} \tag{7}$$

and $\boldsymbol{M}_t$ is the control noise covariance:

$$\boldsymbol{M}_t = \begin{bmatrix} \alpha_1 v_t^2 + \alpha_2 \omega_t^2 & 0 \\ 0 & \alpha_3 v_t^2 + \alpha_4 \omega_t^2 \end{bmatrix} \tag{8}$$

The parameters $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ characterize odometry noise and are typically determined through calibration.

4 Measurement Model Derivation

4.1 Range-Bearing Observations

The measurement model describes how landmark observations relate to the state. For range-bearing sensors (e.g., lidar detecting landmarks):

$$h(\boldsymbol{x}_t, j) = \begin{bmatrix} r_j \\ \phi_j \end{bmatrix} = \begin{bmatrix} \sqrt{(m_{j,x} - x_t)^2 + (m_{j,y} - y_t)^2} \\ \text{atan2}(m_{j,y} - y_t, m_{j,x} - x_t) - \theta_t \end{bmatrix} \tag{9}$$

where r_j is the range to landmark j and ϕ_j is the bearing angle relative to the robot's heading.

4.2 Measurement Jacobian

The measurement Jacobian $\boldsymbol{H}_t$ is crucial for the EKF update. For observation of landmark j:

$$\boldsymbol{H}_t^j = \frac{\partial h}{\partial \boldsymbol{x}} = \begin{bmatrix} \boldsymbol{H}_t^{(r,j)} & \mathbf{0} & \cdots & \boldsymbol{H}_t^{(m,j)} & \cdots & \mathbf{0} \end{bmatrix} \tag{10}$$

where the non-zero blocks are:

$$\boldsymbol{H}_t^{(r,j)} = \frac{\partial h}{\partial \boldsymbol{x}^{(r)}} = \begin{bmatrix} -\frac{\delta_x}{q} & -\frac{\delta_y}{q} & 0 \\ \frac{\delta_y}{q^2} & -\frac{\delta_x}{q^2} & -1 \end{bmatrix} \tag{11}$$

$$\boldsymbol{H}_t^{(m,j)} = \frac{\partial h}{\partial m_j} = \begin{bmatrix} \frac{\delta_x}{q} & \frac{\delta_y}{q} \\ -\frac{\delta_y}{q^2} & \frac{\delta_x}{q^2} \end{bmatrix} \tag{12}$$

where we define:

$$\delta_x = m_{j,x} - x_t \tag{13}$$

$$\delta_y = m_{j,y} - y_t \tag{14}$$

$$q = \sqrt{\delta_x^2 + \delta_y^2} \tag{15}$$

Note that $q = r_j$ is the predicted range to the landmark.

4.3 Measurement Noise

The measurement noise covariance $\boldsymbol{Q}_t$ models sensor uncertainty:

$$\boldsymbol{Q}_t = \begin{bmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\phi^2 \end{bmatrix} \tag{16}$$

where σ_r and σ_ϕ are the standard deviations of range and bearing measurements, respectively.

5 Landmark Initialization

5.1 State Augmentation

When a new landmark is observed for the first time, the state vector must be augmented. Given observation $\boldsymbol{z} = (r, \phi)^\top$:

$$\begin{bmatrix} m_{\text{new},x} \\ m_{\text{new},y} \end{bmatrix} = \begin{bmatrix} x_t + r\cos(\theta_t + \phi) \\ y_t + r\sin(\theta_t + \phi) \end{bmatrix} \tag{17}$$

5.2 Covariance Augmentation

The covariance matrix must also be augmented to include the new landmark's uncertainty. The initialization Jacobians are:

$$\boldsymbol{J}_r = \frac{\partial m_{\text{new}}}{\partial \boldsymbol{x}^{(r)}} = \begin{bmatrix} 1 & 0 & -r\sin(\theta_t + \phi) \\ 0 & 1 & r\cos(\theta_t + \phi) \end{bmatrix} \tag{18}$$

$$\boldsymbol{J}_z = \frac{\partial m_{\text{new}}}{\partial \boldsymbol{z}} = \begin{bmatrix} \cos(\theta_t + \phi) & -r\sin(\theta_t + \phi) \\ \sin(\theta_t + \phi) & r\cos(\theta_t + \phi) \end{bmatrix} \tag{19}$$

The augmented covariance becomes:

$$\boldsymbol{P}_{\text{aug}} = \begin{bmatrix} \boldsymbol{P}_t & \boldsymbol{P}_t \boldsymbol{J}_r^\top \\ \boldsymbol{J}_r \boldsymbol{P}_t & \boldsymbol{J}_r \boldsymbol{P}_{rr} \boldsymbol{J}_r^\top + \boldsymbol{J}_z \boldsymbol{Q}_t \boldsymbol{J}_z^\top \end{bmatrix} \tag{20}$$

6 Implementation Analysis

6.1 Current Implementation Status

Based on analysis of the Python implementation, the repository includes two complete EKF-SLAM variants:

Table 1: Implementation Feature Matrix

Feature	Feature-based	Clustering-based	Complete
Velocity motion model	✓	✓	Yes
Motion Jacobian $\boldsymbol{G}_t$	✓	✓	Yes
Range-bearing measurement model	✓	✓	Yes
Measurement Jacobian $\boldsymbol{H}_t$	✓	✓	Yes
State augmentation	✓	✓	Yes
Covariance augmentation	✓	✓	Yes
Data association (Mahalanobis)	✓	✓	Yes
Corner detection (Split-and-Merge)	✓	×	Yes
Point clustering (DBSCAN)	×	✓	Yes
Occupancy grid mapping	✓	✓	Yes
Loop closure	×	×	No

6.2 Code Analysis: Motion Model

Both implementations use the standard velocity-based motion model with proper Jacobian computation:

```
def predict(self, dx_local, dy_local, dtheta):
    """
    EKF Prediction Step.
    Motion model: robot moves by (dx, dy) in local frame, then rotates by d  .
    """
    theta = self.state[2]
    cos_t, sin_t = np.cos(theta), np.sin(theta)

    # Transform local motion to global frame
    dx_global = dx_local * cos_t - dy_local * sin_t
    dy_global = dx_local * sin_t + dy_local * cos_t

    # Update robot pose
    self.state[0] += dx_global
    self.state[1] += dy_global
    self.state[2] = normalize_angle(self.state[2] + dtheta)

    # Jacobian of motion model w.r.t. state
    n = len(self.state)
    G = np.eye(n)
    G[0, 2] = -dx_local * sin_t - dy_local * cos_t
    G[1, 2] = dx_local * cos_t - dy_local * sin_t

    # Propagate covariance: P = G P  G   + Q_augmented
    self.covariance = G @ self.covariance @ G.T
    self.covariance[:3, :3] += self.Q  # Add process noise to robot pose only
```

Listing 2: Motion Model Implementation

6.3 Code Analysis: Feature Extraction

6.3.1 Split-and-Merge Line Segmentation (Feature-based)

The feature-based implementation uses recursive Split-and-Merge algorithm for line segment detection:

```
def _split_and_merge(self, points, depth=0):
    """Recursive Split-and-Merge for line segment detection."""
    if len(points) < self.min_line_points or depth > 50:
```

```
        return []

    # Fit line to all points
    line_params, p1, p2 = self._fit_line(points)
    if line_params is None:
        return []

    # Find point with maximum distance from line
    max_dist = 0
    max_dist_idx = -1
    for i, pt in enumerate(points):
        d = self._point_to_line_distance(pt, line_params)
        if d > max_dist:
            max_dist = d
            max_dist_idx = i

    # Split if distance exceeds threshold
    if max_dist > self.split_threshold and max_dist_idx > 0:
        left_points = points[:max_dist_idx + 1]
        right_points = points[max_dist_idx:]

        left_segments = self._split_and_merge(left_points, depth + 1)
        right_segments = self._split_and_merge(right_points, depth + 1)

        return left_segments + right_segments
    else:
        # Points fit the line well
        length = np.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)
        if length >= self.min_line_length:
            return [(p1, p2, points)]
        return []
```

Listing 3: Split-and-Merge Line Detection

6.3.2 Corner Detection from Line Intersections

Corners are detected by finding intersections of adjacent line segments with appropriate angles:

```
def detect_corners(self, line_segments):
    """Detect corners from line segment intersections."""
    corners = []
    for i in range(len(line_segments)):
        for j in range(i + 1, len(line_segments)):
            seg1, seg2 = line_segments[i], line_segments[j]

            if not self._segments_are_adjacent(seg1, seg2):
                continue

            angle = self._angle_between_lines(seg1, seg2)
            if 80 <= angle <= 100:  # 90   corner range
                intersection = self._line_intersection(seg1, seg2)
                if intersection:
                    ix, iy = intersection
                    dist = np.sqrt(ix**2 + iy**2)
                    if self.min_range < dist < self.max_range:
                        bearing = np.arctan2(iy, ix)
                        corners.append((dist, bearing))
    return corners
```

Listing 4: Corner Detection from Line Intersections

6.3.3 DBSCAN Clustering (Clustering-based)

The clustering-based implementation uses a DBSCAN-like algorithm for point landmark detection:

```
def _dbscan_cluster(self, points):
    """DBSCAN clustering for point landmark detection."""
    clusters = []
    visited = [False] * len(points)

    for i in range(len(points)):
        if visited[i]:
            continue

        visited[i] = True
        neighbors = self._find_neighbors(points, i, CLUSTER_EPS)

        if len(neighbors) < CLUSTER_MIN_POINTS:
            continue  # Noise point

        cluster = [points[i]]
        queue = neighbors[:]

        while queue:
            j = queue.pop(0)
            if visited[j]:
                continue

            visited[j] = True
            cluster.append(points[j])

            new_neighbors = self._find_neighbors(points, j, CLUSTER_EPS)
            if len(new_neighbors) >= CLUSTER_MIN_POINTS:
                queue.extend(new_neighbors)

        # Validate cluster size and shape
        if self._is_valid_cluster(cluster):
            clusters.append(cluster)

    return clusters
```

Listing 5: DBSCAN Clustering for Point Landmarks

6.4 Code Analysis: Data Association

Both implementations use Mahalanobis distance for robust data association:

```
def associate_landmark(self, z_range, z_bearing):
    """Associate observation to known landmark using Mahalanobis distance."""
    if self.num_landmarks == 0:
        return -1

    robot_x, robot_y, robot_theta = self.state[0], self.state[1], self.state[2]

    best_idx = -1
    best_dist = self.association_threshold

    for i in range(self.num_landmarks):
        lx = self.state[3 + 2 * i]
        ly = self.state[3 + 2 * i + 1]

        # Predicted observation
        dx = lx - robot_x
        dy = ly - robot_y
```

```
        pred_range = np.sqrt(dx**2 + dy**2)
        pred_bearing = normalize_angle(np.arctan2(dy, dx) - robot_theta)

        # Innovation
        innovation = np.array([
            z_range - pred_range,
            normalize_angle(z_bearing - pred_bearing)
        ])

        # Jacobian H for this landmark
        H = self._compute_jacobian_H(i, dx, dy, pred_range)

        # Innovation covariance: S = H P  H   + R
        S = H @ self.covariance @ H.T + self.R

        # Mahalanobis distance: d   =  y    S    y
        try:
            S_inv = np.linalg.inv(S)
            mahal_dist = innovation.T @ S_inv @ innovation
        except np.linalg.LinAlgError:
            continue

        if mahal_dist < best_dist:
            best_dist = mahal_dist
            best_idx = i

    return best_idx
```

Listing 6: Mahalanobis Distance Data Association

6.5 Code Analysis: EKF Update

The EKF update implements the standard Kalman filter equations with numerical safeguards:

```
def ekf_update(self, landmark_idx, z_range, z_bearing):
    """EKF Update Step using observation of a known landmark."""
    robot_x, robot_y, robot_theta = self.state[0], self.state[1], self.state[2]

    # Get landmark position
    lx = self.state[3 + 2 * landmark_idx]
    ly = self.state[3 + 2 * landmark_idx + 1]

    # Predicted observation
    dx = lx - robot_x
    dy = ly - robot_y
    pred_range = np.sqrt(dx**2 + dy**2)
    pred_bearing = normalize_angle(np.arctan2(dy, dx) - robot_theta)

    # Innovation
    innovation = np.array([
        z_range - pred_range,
        normalize_angle(z_bearing - pred_bearing)
    ])

    # Jacobian H
    H = self._compute_jacobian_H(landmark_idx, dx, dy, pred_range)

    # Innovation covariance: S = H P  H   + R
    S = H @ self.covariance @ H.T + self.R

    # Kalman Gain: K = P  H    S
    try:
        K = self.covariance @ H.T @ np.linalg.inv(S)
```

```
    except np.linalg.LinAlgError:
        self.get_logger().warn('Singular matrix in EKF update, skipping')
        return

    # State update with clamping to prevent large jumps
    delta = K @ innovation
    delta[0] = np.clip(delta[0], -0.01, 0.01)  # MAX_POS_DELTA
    delta[1] = np.clip(delta[1], -0.01, 0.01)
    delta[2] = np.clip(delta[2], -0.01, 0.01)  # MAX_THETA_DELTA

    self.state = self.state + delta
    self.state[2] = normalize_angle(self.state[2])

    # Covariance update: P = (I - K H) P
    n = len(self.state)
    I = np.eye(n)
    self.covariance = (I - K @ H) @ self.covariance
```

Listing 7: EKF Update Implementation

6.6 Code Analysis: State Augmentation

Both implementations properly augment the state vector and covariance matrix when adding new landmarks:

```
def add_landmark(self, z_range, z_bearing):
    """Add a new landmark to the state vector."""
    robot_x, robot_y, robot_theta = self.state[0], self.state[1], self.state[2]

    # Compute landmark global position
    global_angle = robot_theta + z_bearing
    lx = robot_x + z_range * np.cos(global_angle)
    ly = robot_y + z_range * np.sin(global_angle)

    # Augment state vector
    self.state = np.append(self.state, [lx, ly])

    # Augment covariance matrix
    n_old = len(self.covariance)
    n_new = n_old + 2

    # Jacobian of landmark initialization w.r.t. robot pose
    cos_a = np.cos(global_angle)
    sin_a = np.sin(global_angle)

    Gp = np.array([
        [1, 0, -z_range * sin_a],
        [0, 1, z_range * cos_a]
    ])

    # Jacobian w.r.t. measurement
    Gz = np.array([
        [cos_a, -z_range * sin_a],
        [sin_a, z_range * cos_a]
    ])

    # New covariance matrix
    P_new = np.zeros((n_new, n_new))
    P_new[:n_old, :n_old] = self.covariance

    # Covariance of new landmark
    P_robot = self.covariance[:3, :3]
    P_lm = Gp @ P_robot @ Gp.T + Gz @ self.R @ Gz.T
```

```
39
40      P_new[n_old:, n_old:] = P_lm
41
42      # Cross-covariance
43      P_new[n_old:, :n_old] = Gp @ self.covariance[:3, :]
44      P_new[:n_old, n_old:] = P_new[n_old:, :n_old].T
45
46      self.covariance = P_new
47      self.num_landmarks += 1
```

Listing 8: State Augmentation Implementation

7 Identified Issues and Recommendations

7.1 Implementation Strengths

The current Python implementation demonstrates several strengths:

1. **Complete EKF-SLAM Implementation:** Both variants include all core EKF-SLAM components with proper Jacobian computations
2. **Robust Data Association:** Mahalanobis distance-based association with appropriate gating thresholds
3. **Numerical Stability:** State update clamping prevents filter divergence from large corrections
4. **Real-time Performance:** Efficient algorithms suitable for real-time operation
5. **Dual Landmark Detection Approaches:** Complementary methods for different environment types

7.2 Areas for Improvement

7.2.1 Process Noise Model Enhancement

Current implementation uses diagonal process noise. A more sophisticated model incorporating velocity-dependent noise would improve accuracy:

$$\boldsymbol{R}_t = \boldsymbol{V}_t \boldsymbol{M}_t \boldsymbol{V}_t^\top \tag{21}$$

where $\boldsymbol{M}_t$ includes velocity-dependent terms for better motion uncertainty modeling.

7.2.2 Loop Closure Detection

The implementation lacks loop closure capabilities. Adding techniques such as:

- Landmark matching across revisited areas
- Pose graph optimization for consistency
- Covariance inflation for landmark re-observations

7.2.3 Advanced Data Association

Current single-best association strategy is conservative but may miss opportunities. Consider:

- Joint Compatibility Branch and Bound (JCBB)
- Multi-hypothesis tracking
- Probabilistic data association

7.3 Minor Enhancements

1. **Adaptive Parameters:** Dynamic adjustment of noise parameters based on motion speed
2. **Landmark Quality Metrics:** Confidence scores for landmark reliability
3. **Outlier Rejection:** More sophisticated innovation gating beyond simple thresholds
4. **Sparsity Exploitation:** For large maps, consider sparse matrix representations

8 Extended Derivations

8.1 Joseph Form Covariance Update

For improved numerical stability, the Joseph form of the covariance update is preferred:

$$\boldsymbol{P}_t = (\boldsymbol{I} - \boldsymbol{K}_t\boldsymbol{H}_t)\bar{\boldsymbol{P}}_t(\boldsymbol{I} - \boldsymbol{K}_t\boldsymbol{H}_t)^\top + \boldsymbol{K}_t\boldsymbol{Q}_t\boldsymbol{K}_t^\top \tag{22}$$

This formulation guarantees positive semi-definiteness of $\boldsymbol{P}_t$ even with numerical errors.

8.2 Mahalanobis Distance for Data Association

The Mahalanobis distance provides a normalized measure of innovation:

$$D_M^2 = (\boldsymbol{z}_t - h(\bar{\boldsymbol{x}}_t))^\top \boldsymbol{S}_t^{-1}(\boldsymbol{z}_t - h(\bar{\boldsymbol{x}}_t)) \tag{23}$$

where $\boldsymbol{S}_t = \boldsymbol{H}_t\bar{\boldsymbol{P}}_t\boldsymbol{H}_t^\top + \boldsymbol{Q}_t$ is the innovation covariance.

Under correct association, D_M^2 follows a chi-squared distribution with degrees of freedom equal to the measurement dimension. A common gating threshold is $\chi^2_{0.95}(2) \approx 5.99$ for 2D measurements.

8.3 Observability Analysis

For EKF-SLAM, the system is not fully observable—only relative positions between the robot and landmarks can be estimated. This manifests as:

- Three unobservable modes corresponding to global translation (x, y) and rotation θ
- The covariance matrix will have (at least) three eigenvalues that grow without bound
- Relative landmark positions remain well-estimated

9 Conclusion

This report has provided a comprehensive analysis of the `EKF_SLAM_ROS2` repository, focusing on its Python implementation with two complementary SLAM approaches. The analysis covers:

1. Complete mathematical derivations for EKF-SLAM
2. Detailed examination of motion and measurement models
3. Implementation analysis of both feature-based and clustering-based variants
4. Code-level examination of core algorithms including data association, EKF updates, and state augmentation
5. Identification of implementation strengths and areas for enhancement

The repository demonstrates a mature and well-implemented EKF-SLAM system suitable for real-world deployment. The dual approach (feature-based with Split-and-Merge corner detection and clustering-based with DBSCAN point detection) provides flexibility for different environments. The implementation includes proper numerical safeguards, robust data association, and complete covariance management.

Future enhancements could focus on loop closure detection, advanced data association techniques, and adaptive parameter tuning to further improve performance in complex environments.

Appendix A: Implementation Details

Key Algorithms Summary

Table 2: EKF-SLAM Implementation Components

Component	**Implementation Details**
Motion Model	Velocity-based with local-to-global transformation
Motion Jacobian	Analytical derivatives with respect to robot pose
Measurement Model	Range-bearing observations
Measurement Jacobian	Analytical derivatives for range and bearing
Data Association	Mahalanobis distance with gating
State Augmentation	Full covariance augmentation with cross-correlations
Feature Extraction	Split-and-Merge (corners) and DBSCAN (points)
Occupancy Mapping	Real-time grid updates with ray tracing
Numerical Stability	State clamping, matrix inversion checks

Parameter Recommendations

Parameter	**Feature-based**	**Clustering-based**
Process Noise Q	[0.01, 0.01, 0.01]	[0.01, 0.01, 0.01]
Measurement Noise R	[0.1, 0.1]	[0.3, 0.3]
Association Threshold	1.0	6.0
Max Landmarks	20	30
Update Limits	$\pm$0.01 m, $\pm$0.01 rad	$\pm$0.05 m, $\pm$0.05 rad

References

[1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*, MIT Press, 2005.

[2] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: Part I," *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99–110, 2006.

[3] T. Bailey and H. Durrant-Whyte, "Simultaneous localization and mapping (SLAM): Part II," *IEEE Robotics & Automation Magazine*, vol. 13, no. 3, pp. 108–117, 2006.